Tutorial: Implement the missing `Snake` game functions (Beginner)

This tutorial is aligned to the current `IGame` interface:

- `Initialize(IPixel[,] pixels)`
- `Update(IPixel[,] pixels)`
- `DrawTitle(IPixel[,] pixels)`
- `HandleInput(ConsoleKey key, ref bool stateChanged)`
- `GetScore()`
- `IsGameOver()`
- `GetGameOverCode()`
- `SetGameOverCode(string? code)`
- `GameId` (property)

Repo-specific note: how the 6-digit claim code works

In this repo, the game **does not mint the 6-digit code itself**.

- Your game sets `gameOver = true` when it ends.
- The outer program (`ConsoleTest/Program.cs`) detects `IsGameOver() == true`, mints a claim code (typically server-generated), then calls `SetGameOverCode(code)`.
- Your game just stores/returns that code via `GetGameOverCode()`.

So: **do not call `CodeGenerator` from inside the game**.

Step 0: What you're building

By the end you will have:

- A snake that moves continuously (speed-gated)
- Food that spawns randomly
- Score that increases when eating food
- Snake gets longer after eating
- Wrap-around walls (default), optional deadly walls
- Game over when hitting yourself (or walls if deadly)
- A red-tinted game-over screen
- A 6-digit claim code that can be displayed after game over (injected via `SetGameOverCode`)

Board size used by the repo code: **20x10** (`pixels[20,10]`).

Step 1: Understand the game variables

Example set of fields you'll need (adjust as desired):

```
// Game variables
private readonly Queue<(int x, int y)> snake = new Queue<(int x, int y)>();
private int snakeLength = 5;

private int headX = 10;
private int headY = 5;

private int directionX = 0;
private int directionY = 1;

private int nextDirectionX = 0;
private int nextDirectionY = 1;

private readonly Random rand = new Random();
private (int x, int y) food;

private int score = 0;
private float rainbowShift = 0f;

private bool gameOver = false;
private bool wallsAreDeadly = false;
private int moveCounter = 0;

// Code minted by Program.cs after game over; may be null until set.
private string? gameOverCode;

// Required by IGame (Program uses this when minting a claim code).
public string GameId { get; } = "REPLACE_WITH_REAL_GAME_ID";
```

Step 2: Implement `DrawTitle(IPixel[,] pixels)`

`Program.cs` calls `DrawTitle` while on the title screen. Keep it simple: fill the board with a pattern and optionally draw a letter/logo.

Minimal implementation idea:

- Animate a background (optional).
- Draw a simple 'S' or 'Snake' shape in black pixels on top.

Step 3: Implement `Initialize(IPixel[,] pixels)`

`Initialize` must reset the game state and create the starting snake.

```
public void Initialize(IPixel[,] pixels)
{
    snake.Clear();

    snakeLength = 5;
```

```
        headX = 10;
        headY = 5;

        directionX = 0;
        directionY = 1;
        nextDirectionX = 0;
        nextDirectionY = 1;

        score = 0;
        gameOver = false;
        moveCounter = 0;

        // Reset the injected claim code for a new run.
        gameOverCode = null;

        // Spawn initial food
        food = (rand.Next(20), rand.Next(10));

        // Create initial snake body (behind the head)
        for (int i = 0; i < snakeLength; i++)
        {
            snake.Enqueue((headX, headY - i));
        }
    }
```

Step 4: Implement `Update(IPixel[,] pixels)`

Responsibilities:

- If `gameOver`, draw game-over visuals and stop advancing gameplay
- Gate movement with `moveCounter` and `GetMoveSpeed()`
- Apply buffered direction
- Move the head
- Handle walls (wrap vs deadly)
- Detect self collision
- Detect food, grow, increment score, respawn food
- Update queue body
- Draw frame

Important: when game over occurs, set `gameOver = true` and return. Do **not** generate a claim code here.

```
    public void Update(IPixel[,] pixels)
    {
        if (gameOver)
        {
            DrawGameOver(pixels);
            return;
        }
```

```
        moveCounter++;

        if (moveCounter < GetMoveSpeed())
        {
            DrawGame(pixels);
            return;
        }

        moveCounter = 0;

        directionX = nextDirectionX;
        directionY = nextDirectionY;

        headX += directionX;
        headY += directionY;

        if (wallsAreDeadly)
        {
            if (headX < 0 || headX >= 20 || headY < 0 || headY >= 10)
            {
                gameOver = true;
                DrawGameOver(pixels);
                return;
            }
        }
        else
        {
            if (headX >= 20) headX = 0;
            if (headX < 0) headX = 19;
            if (headY >= 10) headY = 0;
            if (headY < 0) headY = 9;
        }

        if (snake.Contains((headX, headY)))
        {
            gameOver = true;
            DrawGameOver(pixels);
            return;
        }

        if (headX == food.x && headY == food.y)
        {
            score++;
            snakeLength++;

            do
            {
                food = (rand.Next(20), rand.Next(10));
            } while (snake.Contains(food) || (food.x == headX && food.y == headY));
        }

        snake.Enqueue((headX, headY));
        if (snake.Count > snakeLength)
        {
```

```
            snake.Dequeue();
        }

        DrawGame(pixels);
    }
```

Step 5: Implement `HandleInput(ConsoleKey key, ref bool stateChanged)`

- Movement keys should update `nextDirectionX/nextDirectionY`
- Prevent instant reversal (don't allow the opposite direction)
- Escape should set `stateChanged = true` (Program uses this to return to title)

```
public void HandleInput(ConsoleKey key, ref bool stateChanged)
{
    if (gameOver)
    {
        if (key == ConsoleKey.Escape)
        {
            stateChanged = true;
        }
        return;
    }

    switch (key)
    {
        case ConsoleKey.W:
        case ConsoleKey.UpArrow:
            if (directionX != 1)
            {
                nextDirectionX = -1;
                nextDirectionY = 0;
            }
            break;

        case ConsoleKey.S:
        case ConsoleKey.DownArrow:
            if (directionX != -1)
            {
                nextDirectionX = 1;
                nextDirectionY = 0;
            }
            break;

        case ConsoleKey.A:
        case ConsoleKey.LeftArrow:
            if (directionY != 1)
            {
                nextDirectionX = 0;
                nextDirectionY = -1;
```

```
            }
            break;

        case ConsoleKey.D:
        case ConsoleKey.RightArrow:
            if (directionY != -1)
            {
                nextDirectionX = 0;
                nextDirectionY = 1;
            }
            break;

        case ConsoleKey.Escape:
            stateChanged = true;
            break;
    }
}
```

Step 6: Implement `GetScore`, `IsGameOver`, `GetGameOverCode`, `SetGameOverCode`

These must match `IGame` exactly.

```
public int GetScore() => score;

public bool IsGameOver() => gameOver;

public string? GetGameOverCode() => gameOverCode;

public void SetGameOverCode(string? code)
{
    gameOverCode = code;
}
```

Step 7: Implement `GetMoveSpeed()`

Lower number = faster movement.

```
private int GetMoveSpeed()
{
    if (score < 5) return 3;
    if (score < 10) return 2;
    return 1;
}
```

Step 8: Implement `DrawGame` + `DrawGameOver`

Basic drawing scheme:

- Background: dark
- Snake body: green
- Snake head: brighter green
- Food: red
- Game over: draw final frame + red overlay

```
private void DrawGame(IPixel[,] pixels)
{
    for (sbyte i = 0; i < 20; i++)
    {
        for (sbyte j = 0; j < 10; j++)
        {
            pixels[i, j] = new Pixel(20, 20, 40);
        }
    }

    foreach (var (x, y) in snake.Take(Math.Max(0, snake.Count - 1)))
    {
        pixels[x, y] = new Pixel(0, 180, 0);
    }

    if (snake.Count > 0)
    {
        var head = snake.Last();
        pixels[head.x, head.y] = new Pixel(0, 255, 0);
    }

    pixels[food.x, food.y] = new Pixel(255, 0, 0);
}

private void DrawGameOver(IPixel[,] pixels)
{
    DrawGame(pixels);

    for (int i = 0; i < 20; i++)
    {
        for (int j = 0; j < 10; j++)
        {
            pixels[i, j] = new Pixel(150, 0, 0);
        }
    }
}
```